

INTERNATIONAL UNIVERSITY

IT096IIU – NETCENTRIC

LAB 01 – GO FOUNDATION

FOOD ORDERING SYSTEM

Phạm Gia Thịnh - ITITI24061
Huỳnh Ngọc Thủy Tiên - ITITI24065

1 HOMEWORK EXTENSION

1.1 Short Design Note

To handle the new customer and order types, we declared new custom string types (`CustomerType` and `OrderType`) and defined constants for the specific categories (Regular/Member and Pickup/Delivery).

```
type CustomerType string

const (
    Regular CustomerType = "Regular"
    Member  CustomerType = "Member"
)

type OrderType string

const (
    Pickup  OrderType = "Pickup"
    Delivery OrderType = "Delivery"
)
```

We also established two core constants for the business rules:

```
const (
    memberDiscountPercent = 5
    deliveryFee           = 30000
)
```

We then updated the `Order` struct to include these new types:

```
type Order struct {
    OrderID      string
    Items        map[string]int
    CustomerType CustomerType
    OrderType    OrderType
}
```

Explanation: The main idea is that we specify the type of customer and order when making a new order. Then, inside the `printOrderSummary` function, we check this data. If `CustomerType == "Member"`, it triggers the discount calculation.

1.2 Handling the Requirement Change

To implement the new business rules cleanly, we created two helper functions: `calculateDiscount` and `calculateDeliveryFee`.

```
func calculateDiscount(customerType CustomerType, foodSubtotal int) int {
    if customerType == Member {
        return foodSubtotal * memberDiscountPercent / 100
    }
    return 0
}
```

```
func calculateDeliveryFee(orderType OrderType) int {
    if orderType == Delivery {
        return deliveryFee
    }
    return 0
}
```

Explanation: The new `printOrderSummary` now utilizes two new variables: `discount` and `fee`. The function still contains the main logic for calculating the raw food subtotal, but `discount` and `fee` alter the `finalTotal`. Because the discount is calculated *before* the delivery fee is added, it correctly applies only to the food.

For example, if a user buys items for 100,000 VND, is a Member, and chooses Delivery:

- **Discount:** 5,000 VND (5% of 100,000 VND)
- **Delivery Fee:** 30,000 VND
- **Final Total:** $100,000 - 5,000 + 30,000 = 125,000$ VND.

```

func printOrderSummary(order Order, inventory map[string]*FoodItem) {
    fmt.Printf("\n== Order Summary (%s) ==\n", order.OrderID)
    fmt.Printf("Customer: %s | Order type: %s\n", order.CustomerType, order.OrderType)

    if len(order.Items) == 0 {
        fmt.Println("Order is empty.")
        return
    }

    for itemID, qty := range order.Items {
        food := inventory[itemID]
        fmt.Printf("- %-17s | Qty: %d x %d VND = %d VND\n", food.Name, qty, food.Price, food.Price*qty)
    }

    subtotal := calculateSubtotal(order, inventory)
    discount := calculateDiscount(order.CustomerType, subtotal)
    fee := calculateDeliveryFee(order.OrderType)
    finalTotal := subtotal - discount + fee

    fmt.Println("-----")
    fmt.Printf("Subtotal:      %d VND\n", subtotal)
    fmt.Printf("Discount:     -%d VND\n", discount)
    fmt.Printf("Delivery fee:  %d VND\n", fee)
    fmt.Printf("Final Total:  %d VND\n\n", finalTotal)
}

```

1.3 Test Cases & Results

We ran test cases to verify all combinations of the new business rules.

Test Case	Expected Behavior	Result
Regular & Pickup	No discount, no delivery fee. Final Total = Subtotal.	Passed
Member & Pickup	5% discount on food, no delivery fee.	Passed
Regular & Delivery	No discount, +30,000 VND delivery fee.	Passed
Member & Delivery	5% discount on food only, +30,000 VND delivery fee.	Passed

1.4 AI Collaboration Log (Homework)

#	What we asked AI	What AI suggested	What we changed/verified
1	Asked AI to review our homework code for applying the new Customer and Order types.	Confirmed our struct design was excellent but pointed out that we accidentally set the member discount to 10% instead of 5%.	We verified the lab requirements, updated the constant to <code>memberDiscountPercent = 5</code> , and corrected our calculation examples.